



Producto 3.

[Sirviendo datos con GraphQL y MongoDB]

FullStackAttack

Erick Coll Rodríguez
Carles Miguel Millán
Jacobo Barrera Toba



FP.450 - (P) Des. full stack de sol. web JavaScript y serv. web



Tabla de contenido

1. Introducción	4
1.1. Propósito de este documento.....	4
1.2. A quién va dirigido	4
1.3. Cómo usar esta guía.....	4
1.4. Producto 3 de un vistazo.....	5
Objetivos didácticos según enunciado.....	5
1.5. Stack tecnológico elegido	5
1.6. El equipo FullStackAttack.....	7
2. Prerequisitos e instalación de herramientas	7
2.1. Resumen de herramientas requeridas	7
2.2. Instalación de Node.js.....	8
Windows	8
macOS	8
Verificación (común a ambos sistemas)	8
2.3. Instalación de Docker Desktop	9
Windows	9
macOS	9
Verificación (común a ambos sistemas)	10
2.4. Instalación de Git	10
Windows	10
macOS	10
Configuración inicial obligatoria (común a ambos sistemas)	10
2.5. Instalación de Visual Studio Code.....	11
Windows	11
macOS	11
2.6. Extensiones recomendadas para VS Code.....	12
2.7. Instalación de Postman.....	12
Windows y macOS	12
2.8. Verificación final del entorno.....	13
3. Repositorio GitHub.....	13
3.1. URL del proyecto.....	13
3.2. Acceso y aceptación de la invitación	14
3.3. Clonar el proyecto en local	14
Ruta recomendada.....	14

Comandos de clonado (Windows — PowerShell).....	14
Comandos de clonado (macOS — Terminal)	15
Primera autenticación.....	15
3.4. Verificación post-clonado	15
4. Estructura del proyecto	15
4.1. Vista general del árbol de ficheros	16
4.2. Archivos de configuración.....	16
4.2.1. package.json.....	16
4.2.2. .gitignore.....	17
4.2.3. .env.example y .env	18
4.2.4. docker-compose.yml.....	18
4.3. Estructura de src/	19
5.1. Instalar dependencias del proyecto.....	20
5.2. Configurar el fichero .env	20
Windows (PowerShell).....	20
macOS (Terminal).....	20
5.3. Generar un JWT_SECRET único.....	21
5.4. Levantar MongoDB con Docker Compose	21
5.5. Acceder al panel Mongo Express	22
5.6. Comandos útiles del día a día	22
6. Flujo de trabajo en equipo (GitFlow)	23
6.1. Estrategia de ramas	23
6.2. Reglas de oro del equipo.....	23
6.3. Flujo diario recomendado	24
6.4. Convenciones de commits (Conventional Commits)	24
6.5. Apertura de Pull Requests	25
7. Estado actual del proyecto.....	26
7.1. Fase 1 — Infraestructura (COMPLETADA)	26
7.2. Fase 2 — Backend inicial con Express, Apollo y MongoDB (COMPLETADA).....	26
7.3. Fase 2.5 — Refuerzo y profesionalización del backend (COMPLETADA).....	27
7.4. Fase 3 — CRUD en memoria de las tres entidades (COMPLETADA).....	27
7.5. Fase 4 — Persistencia real en MongoDB (COMPLETADA)	28
7.6. Fase 5 — Autenticación JWT del administrador (COMPLETADA)	30
7.7. Fix del enmascarado de errores en Apollo (PR #10)	32
7.8. Colección Postman y tests automatizados (PR #11)	33

7.9. Despliegue: MongoDB Atlas y CodeSandbox (PR #12)	34
7.10. Historial de Pull Requests del proyecto	36
7.11. Acuerdo del equipo: main congelada hasta la entrega	37
7.12. Información de la entrega.....	37
8. Troubleshooting — Problemas comunes.....	38
8.1. "Docker daemon is not running" al ejecutar docker ps.....	38
8.2. Puerto 27017 ya en uso al levantar MongoDB	39
8.3. npm install falla con errores de permisos en macOS.....	39
8.4. "fatal: not a git repository" al clonar	39
8.5. VS Code no reconoce el comando "code" en la terminal	39
8.6. "Permission denied" al pushear a GitHub.....	39
8.7. Conflictos al hacer merge de develop en la rama personal.....	40
8.8. MongoDB Express no carga en el navegador.....	40
9. Checklist de verificación final.....	40
9.1. Instalaciones	40
9.2. Repositorio y proyecto.....	41
9.3. Docker y base de datos	41
9.4. GitFlow	42
10. Recursos y referencias	42
10.1. Documentación oficial	43
10.2. Convenciones y buenas prácticas	43
10.3. Enlaces internos del proyecto.....	43
11. Prompts de IA utilizados	43
11.1. Filosofía de uso de la IA en este proyecto	43
11.2. Tabla de prompts del equipo	44
11.3. Consideraciones finales sobre el uso de IA.....	47

1. Introducción

1.1. Propósito de este documento

Este documento es la guía técnica oficial del Producto 3 de la asignatura FP.450, desarrollado por el equipo FullStackAttack. Ha sido redactado con tres objetivos simultáneos en mente, y está diseñado para ser útil para cada uno de ellos sin necesidad de documentos adicionales.

1. Servir de manual operativo a los miembros del equipo. Cualquier integrante debe poder partir de un ordenador limpio, seguir esta guía paso a paso, y llegar al mismo estado del proyecto que el resto. Las instrucciones cubren tanto Windows 10/11 como macOS (Intel y Apple Silicon).
2. Constituir una pieza de entrega académica. Recoge de forma ordenada y justificada las decisiones técnicas tomadas, la infraestructura montada, el flujo de trabajo adoptado y el estado del proyecto en el momento de la entrega. Permite al consultor evaluar el trabajo realizado sin necesidad de ejecutar el proyecto.
3. Funcionar como guion del vídeo explicativo requerido en la rúbrica. La estructura de capítulos, los comandos exactos y las capturas conceptuales permiten grabar la exposición sin improvisación, asegurando que no se olvide ningún aspecto evaluable.

1.2. A quién va dirigido

El documento tiene dos audiencias principales con perfiles técnicos distintos, y ha sido redactado buscando el equilibrio entre ambas:

- **Compañeros del equipo:** desarrolladores con conocimientos de JavaScript pero sin experiencia previa en Node.js backend, GraphQL o MongoDB. Se asume familiaridad con la línea de comandos y con Git, pero se explican los conceptos clave cuando aparecen por primera vez.
- **Consultor de la asignatura:** evaluador que juzgará el cumplimiento de la rúbrica. Se incluyen justificaciones de las decisiones técnicas tomadas, referencias cruzadas con los criterios de evaluación, y evidencias de buenas prácticas (GitFlow, Conventional Commits, separación de configuración y código, etc.).

1.3. Cómo usar esta guía

La guía está diseñada para leerse de forma secuencial la primera vez, y después como documento de referencia. Los capítulos 2 al 5 describen el proceso de setup paso a paso, y se recomienda seguirlos en orden. El capítulo 6 describe el flujo de trabajo diario en equipo. Los capítulos 7 al 10 son de consulta puntual.

Convenciones tipográficas

Los bloques de código en fondo gris contienen comandos que se copian y ejecutan tal cual en la terminal.

Las cajas verdes contienen consejos prácticos (tips).

Las cajas ámbar contienen advertencias importantes que hay que leer antes de continuar.

Las cajas azules aportan contexto o información adicional.

Las cajas rojas marcan acciones prohibidas o errores críticos a evitar.

1.4. Producto 3 de un vistazo

El Producto 3 es la tercera iteración del proyecto JobConnect, una aplicación web para la gestión de ofertas y demandas de empleo. En esta fase se migra toda la lógica de persistencia que en el Producto 2 residía en el navegador (localStorage e IndexedDB) a un backend real compuesto por una API GraphQL servida por Express sobre Apollo Server, y una base de datos NoSQL MongoDB.

Es importante destacar que el Producto 3 NO contiene frontend. Toda la validación funcional se realiza mediante Postman. La integración con la interfaz del Producto 2 se abordará en el Producto final.

Objetivos didácticos según enunciado

- Conocer y aplicar Node.js y Express como tecnologías de programación de backend en JavaScript.
- Desarrollar un servicio web funcional con Express.
- Conocer y aplicar la especificación GraphQL como punto de acceso API de aplicaciones para la gestión de información con bases de datos.
- Conocer y utilizar MongoDB como gestor de bases de datos NoSQL, sin Mongoose, empleando el driver nativo oficial.
- Realizar un CRUD completo con GraphQL con acceso a MongoDB.

1.5. Stack tecnológico elegido

A continuación se resume la pila técnica adoptada, junto con la justificación que condujo a cada decisión. Cada elección responde tanto al enunciado como a criterios de la rúbrica que apuntan hacia la máxima puntuación.

Capa	Tecnología	Justificación
Runtime	Node.js 20 LTS +	Requerido por el enunciado. Se fija versión mínima 20 para soportar sintaxis moderna y dependencias actuales.
Framework web	Express 5	Recomendado explícitamente en el enunciado. Se usa como "base" para montar Apollo Server, no como framework completo.
API	Apollo Server 5	Implementación de referencia de GraphQL, recomendada por el material docente de la UOC. Proporciona Apollo Sandbox para testing.
GraphQL	graphql 16	Motor de ejecución estándar de GraphQL. Dependencia obligatoria de Apollo Server.
Base de datos	MongoDB 7	NoSQL orientada a documentos JSON, natural para persistir estructuras heredadas del Producto 2. Requerida por el enunciado.
Driver BBDD	mongodb (oficial)	Driver nativo de MongoDB. Se evita Mongoose expresamente (lo pide el enunciado). Permite trabajar directamente con la API de colecciones y documentos.
Autenticación	jsonwebtoken	Emisión y verificación de JSON Web Tokens para el login del administrador. Estándar de la industria. La rúbrica valora "autenticación avanzada".
Hashing de claves	bcryptjs	Función de derivación de claves segura para almacenar contraseñas. Nunca se guardan en texto plano.
Configuración	dotenv	Carga variables de entorno desde .env. Separa configuración del código (principio 12-factor app).
CORS	cors	Permite que Postman y clientes externos consuman la API sin restricciones de origen durante el desarrollo.
Contenedores	Docker + Compose	Levanta MongoDB y Mongo Express de forma reproducible, sin instalaciones invasivas en la máquina local. Requerido para la nota máxima.
Dev tools	nodemon	Reinicia automáticamente el servidor al guardar cambios durante el desarrollo. Solo dependencia de desarrollo.
Testing API	Postman	Herramienta recomendada en el enunciado para validar la API sin frontend.
Control de versiones	Git + GitHub	Requerido por el enunciado. Flujo GitFlow simplificado con ramas main/develop/feature.

1.6. El equipo FullStackAttack

El desarrollo del Producto 3 corre a cargo de un equipo de tres personas que se reparten responsabilidades de implementación, documentación y testing. Las decisiones técnicas se toman de forma consensuada, y el repositorio se gestiona siguiendo un flujo GitFlow que garantiza que el trabajo de cada miembro no interfiere con el del resto.

Integrante	Rama principal de trabajo	GitHub
Erick Coll	feature/erick	ErickColl
Carles Miguel	feature/carles	Carles Miguel Millán
Jacobo	feature/jacobo	Jacobo Barrera Toba

2. Prerequisitos e instalación de herramientas

Antes de tocar código, cada miembro del equipo debe preparar su máquina con el conjunto de herramientas que describe este capítulo. El orden recomendado de instalación es el que sigue, ya que algunas dependencias se benefician de tener otras instaladas previamente (por ejemplo, Git debe estar disponible antes de clonar el repositorio).

Todas las instrucciones se presentan diferenciadas para Windows 10/11 y para macOS (Intel y Apple Silicon). Si una sección no especifica el sistema operativo, las indicaciones aplican a ambos.

2.1. Resumen de herramientas requeridas

Herramienta	Versión mínima	Finalidad
Node.js	20.x LTS	Runtime de JavaScript para ejecutar el backend.
npm	Incluido	Gestor de paquetes (viene con Node).
Docker Desktop	20.x o superior	Contenerización de MongoDB y Mongo Express.
Git	2.40 o superior	Control de versiones.
VS Code	Última estable	Editor principal del equipo.
Postman	Última estable	Testing de la API GraphQL.

2.2. Instalación de Node.js

Node.js es el intérprete de JavaScript del servidor. Instalarlo también deja disponible npm, el gestor de paquetes que usaremos para descargar las dependencias del proyecto.

Windows

1. Descargar el instalador oficial desde nodejs.org:

```
https://nodejs.org/en/download
```

2. Seleccionar la versión LTS (v22.x o superior) y pulsar “Windows Installer (.msi)”.
3. Ejecutar el instalador descargado. Aceptar todos los valores por defecto excepto la pantalla “Tools for Native Modules”:

Pantalla “Tools for Native Modules”

DESMARCAR la casilla “Automatically install the necessary tools...”. Activarla dispara una instalación adicional de Python y Visual Studio Build Tools de más de 20 minutos que no necesitamos para este proyecto.

4. Completar la instalación y cerrar cualquier terminal abierta antes de verificar.

macOS

En macOS recomendamos usar Homebrew, el gestor de paquetes estándar de la plataforma. Si todavía no se tiene Homebrew instalado, instalarlo primero con una sola orden en el Terminal:

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Una vez Homebrew esté disponible, instalar Node.js:

```
brew install node@22  
brew link --overwrite node@22
```

Alternativa sin Homebrew en macOS

También es válido descargar el instalador .pkg desde <https://nodejs.org>. Funciona idéntico al .msi de Windows: doble clic y siguiente-siguiente-terminar.

Verificación (común a ambos sistemas)

Abrir una terminal NUEVA (cualquier ventana anterior no detectará los binarios recién instalados) y ejecutar:

```
node -v
npm -v
```

La salida esperada debe mostrar una versión de Node igual o superior a v20.x.x y una versión de npm igual o superior a 10.x.x. Si ambos comandos responden con una versión, la instalación ha sido correcta.

2.3. Instalación de Docker Desktop

Docker nos permite levantar MongoDB y su panel web sin instalarlos directamente en la máquina. Toda la base de datos vive dentro de contenedores aislados que podemos encender, apagar o destruir sin afectar al resto del sistema.

Windows

1. Descargar Docker Desktop for Windows desde el sitio oficial:

```
https://www.docker.com/products/docker-desktop/
```

2. Ejecutar el instalador Docker Desktop Installer.exe. Aceptar la instalación de WSL 2 cuando lo solicite (es obligatorio en Windows 10/11).
3. Reiniciar el equipo cuando lo pida el instalador.
4. Abrir Docker Desktop desde el menú Inicio. La primera vez tarda 1-2 minutos en arrancar completamente. El icono de la ballena en la bandeja del sistema debe quedarse estable, sin animación.
5. Aceptar los términos de uso cuando aparezca el diálogo.

Requisitos de hardware en Windows

Docker Desktop en Windows requiere que la virtualización esté habilitada en la BIOS/UEFI (Intel VT-x o AMD-V). En la mayoría de portátiles modernos ya viene activada. Si al arrancar Docker aparece un error de virtualización, revisar esta opción en la configuración de la BIOS.

macOS

Si el Mac es Apple Silicon (M1, M2, M3, M4), descargar la versión Apple Chip. Si es Intel, descargar la versión Intel Chip.

1. Descargar Docker Desktop for Mac desde:

```
https://www.docker.com/products/docker-desktop/
```

2. Abrir el archivo Docker.dmg descargado y arrastrar el icono de Docker a la carpeta Applications.
3. Lanzar Docker desde el Launchpad o la carpeta Aplicaciones.

4. Autorizar los permisos de administrador que solicite. Aceptar los términos de uso.
5. Esperar a que el icono de la ballena en la barra superior se quede estable.

Como alternativa, también se puede instalar con Homebrew:

```
brew install --cask docker
```

Verificación (común a ambos sistemas)

```
docker --version
docker compose version
docker ps
```

Los tres comandos deben devolver una versión y, en el caso de `docker ps`, una cabecera vacía. Si `docker ps` devuelve un error tipo "Cannot connect to the Docker daemon", Docker Desktop aún no ha terminado de arrancar — esperar medio minuto y volver a probar.

2.4. Instalación de Git

Git es el sistema de control de versiones que usamos para coordinar el código entre los tres miembros del equipo mediante GitHub.

Windows

Descargar el instalador oficial desde:

```
https://git-scm.com/download/win
```

Ejecutar el .exe descargado y aceptar los valores por defecto en todas las pantallas. El instalador de Git para Windows incluye también Git Bash, una terminal compatible con comandos Unix que puede resultar útil para quien venga de macOS/Linux.

macOS

macOS trae Git de serie, pero puede ser una versión antigua. Para instalar una versión reciente:

```
brew install git
```

También se puede obtener Git instalando las Command Line Tools de Xcode, que se pedirán automáticamente la primera vez que se ejecute git en el Terminal:

```
xcode-select --install
```

Configuración inicial obligatoria (común a ambos sistemas)

Antes del primer commit, Git necesita saber quién hace los cambios. Ejecutar una única vez por máquina, sustituyendo los valores entre comillas por los propios:

```
git config --global user.name "Nombre Apellido"
git config --global user.email "tu-email@ejemplo.com"
```

Usar el mismo email de GitHub

El email configurado aquí debe coincidir exactamente con el email verificado en la cuenta de GitHub. En caso contrario, los commits aparecerán en el historial como "desconocidos" y no se atribuirán correctamente al usuario.

2.5. Instalación de Visual Studio Code

Visual Studio Code (VS Code) es el editor elegido por el equipo. Es gratuito, multiplataforma, ligero, y tiene un ecosistema de extensiones amplio que facilita el trabajo con Node.js, GraphQL, Docker y MongoDB.

Windows

Descargar e instalar desde:

```
https://code.visualstudio.com/Download
```

En la pantalla “Seleccionar tareas adicionales” del instalador, es muy conveniente marcar estas tres casillas:

- Agregar la acción "Abrir con Code" al menú contextual del archivo.
- Agregar la acción "Abrir con Code" al menú contextual del directorio.
- Agregar a PATH (normalmente ya viene marcada).

macOS

Descargar el .zip desde el enlace anterior, descomprimirlo, y arrastrar Visual Studio Code.app a la carpeta Applications. Alternativamente con Homebrew:

```
brew install --cask visual-studio-code
```

En ambos sistemas, una vez abierto VS Code, pulsando Cmd+Shift+P (macOS) o Ctrl+Shift+P (Windows) y buscando "Shell Command: Install 'code' command in PATH" queda disponible el comando "code" desde el terminal para abrir carpetas rápido.

2.6. Extensiones recomendadas para VS Code

El equipo adopta un set de extensiones común para que el entorno de desarrollo sea homogéneo entre los tres miembros. Esto evita discrepancias de formato, facilita la detección de errores y permite aprovechar al máximo las capacidades del editor sobre el stack técnico elegido.

Extensión	Publicador	Función
GraphQL: Syntax Highlighting	GraphQL Foundation	Resaltado sintáctico para ficheros .graphql y literales gql.
GraphQL: Language Feature Support	GraphQL Foundation	Autocompletado y validación del schema.
ESLint	Microsoft	Análisis estático de JavaScript. Detecta errores antes de ejecutar.
Prettier — Code formatter	Prettier	Formateo automático uniforme del código.
MongoDB for VS Code	MongoDB	Explorador y consultor de MongoDB integrado en el editor.
Docker	Microsoft	Gestión visual de contenedores, imágenes y volúmenes.

Todas se instalan desde la pestaña Extensiones de VS Code (icono de piezas de puzzle en la barra lateral o atajo Cmd+Shift+X en macOS / Ctrl+Shift+X en Windows).

2.7. Instalación de Postman

Postman es un cliente HTTP orientado al desarrollo y testing de APIs. En este proyecto lo usaremos intensivamente para enviar consultas y mutaciones GraphQL al backend, ya que no hay frontend durante el Producto 3.

Windows y macOS

Descargar desde el sitio oficial:

<https://www.postman.com/downloads/>

La instalación es un asistente convencional en ambos sistemas. La primera vez abrirá una pantalla de login; es opcional crear cuenta (útil si se quieren sincronizar colecciones entre dispositivos, pero no imprescindible para el Producto 3).

Alternativa sin instalación: Apollo Sandbox

Apollo Server incorpora una interfaz web llamada Apollo Sandbox, accesible desde el navegador una vez el servidor esté corriendo. Aunque Postman es la herramienta principal exigida por el enunciado, Apollo Sandbox resulta útil durante el desarrollo para probar queries rápidamente sin salir del navegador.

2.8. Verificación final del entorno

Una vez completadas todas las instalaciones anteriores, abrir una terminal (PowerShell en Windows o Terminal en macOS) y ejecutar esta batería de comandos para confirmar que todo está operativo:

```
node -v
npm -v
docker --version
docker compose version
git --version
code --version
```

La salida debe incluir una versión para cada uno. Si alguno falla con el mensaje "command not found" o similar, volver al apartado correspondiente y revisar la instalación.

Verificación exitosa de referencia

```
node -v    → v22.15.0 (o superior)
npm -v     → 11.x.x
docker --version → Docker version 28.x.x
git --version → git version 2.x
code --version → 1.9x.x
```

3. Repositorio GitHub

3.1. URL del proyecto

Todo el código, documentación y configuración del Producto 3 vive en un repositorio nuevo, independiente del Producto 2, tal como exige el enunciado. La URL oficial es:

```
https://github.com/ErickColl/FullStackAttack-Producto3
```

 Por qué un repositorio nuevo

El enunciado lo especifica literalmente: "Para cada producto se tendrá que crear un github independiente, nunca se trabajará en el mismo para que de tiempo al consultor de corregir el trabajo sin que se vaya reemplazando el código con el siguiente producto." Mantener repos separados por entrega garantiza que cada versión entregada quede congelada para su evaluación.

3.2. Acceso y aceptación de la invitación

El propietario del repositorio (Erick) añade a cada miembro del equipo y al consultor como colaboradores desde la sección Settings → Collaborators de la web de GitHub. Los invitados reciben un email automático de GitHub y deben aceptar la invitación antes de poder clonar o pushear.

Pasos para aceptar la invitación:

1. Abrir el email recibido de noreply@github.com con asunto similar a "ErickColl invited you to ErickColl/FullStackAttack-Producto3".
2. Pulsar el enlace "View invitation". GitHub abrirá una página de aceptación.
3. Pulsar "Accept invitation".
4. GitHub redirige al repositorio, ya con permisos de escritura.

3.3. Clonar el proyecto en local

Una vez aceptada la invitación, cada miembro debe clonar el repositorio a una ubicación de su sistema de ficheros. El equipo usa por convención una carpeta sin espacios ni caracteres especiales para evitar problemas con rutas en Node.

Ruta recomendada

Sistema operativo	Ruta sugerida
Windows	C:\Users\<TuUsuario>\Desktop\Producto-3-FullStack
macOS	/Users/<TuUsuario>/Desktop/Producto-3-FullStack

Comandos de clonado (Windows — PowerShell)

```
cd $HOME\Desktop
mkdir Producto-3-FullStack -ErrorAction SilentlyContinue
cd Producto-3-FullStack
git clone https://github.com/ErickColl/FullStackAttack-Producto3.git .
```

Comandos de clonado (macOS — Terminal)

```
cd ~/Desktop
mkdir -p Producto-3-FullStack
cd Producto-3-FullStack
git clone https://github.com/ErickColl/FullStackAttack-Producto3.git .
```

⚠ El punto al final del git clone

El punto "." al final del comando git clone es intencional. Indica a Git que debe clonar el contenido del repositorio dentro de la carpeta actual, sin crear una subcarpeta adicional. Sin ese punto, el código acabaría en Producto-3-FullStack/FullStackAttack-Producto3, con un nivel de anidamiento innecesario.

Primera autenticación

La primera vez que cada miembro clonee o haga push, Git pedirá autenticarse contra GitHub. En sistemas modernos se abre el navegador automáticamente, se inicia sesión en la cuenta de GitHub, se autoriza el acceso y se vuelve a la terminal. Git guarda credenciales para operaciones futuras mediante el Credential Manager del sistema (Windows) o el Keychain (macOS).

3.4. Verificación post-clonado

Tras clonar, comprobar que la rama activa es develop y que el repositorio refleja los últimos cambios:

```
cd Producto-3-FullStack
git status
git branch -a
```

La salida esperada es una rama local inicial (main o develop según el orden de clonado), junto con la lista completa de ramas remotas: remotes/origin/main, remotes/origin/develop, remotes/origin/feature/erick, remotes/origin/feature/carles, remotes/origin/feature/jacobo.

4. Estructura del proyecto

Tras clonar el repositorio, la carpeta local contiene la estructura que se detalla a continuación. Conocer qué hace cada fichero y cada carpeta es imprescindible para mantener el orden conforme avanza el desarrollo y para saber dónde añadir nuevos ficheros cuando se implementen las próximas funcionalidades.

4.1. Vista general del árbol de ficheros

```
FullStackAttack-Producto3/
├── .env.example      ← plantilla de variables (versionada)
├── .env              ← variables reales (NO se sube a Git)
├── .gitignore        ← reglas de exclusión para Git
├── docker-compose.yml ← infraestructura MongoDB + Mongo Express
├── package.json      ← metadatos y dependencias del proyecto
├── package-lock.json ← versiones exactas instaladas
├── node_modules/     ← dependencias (ignorado por Git)
├── docs/             ← documentación, prompts IA, mapa conceptual
├── postman/          ← colección Postman para el evaluador
└── src/              ← código fuente del backend
    ├── config/       ← conexión BD y carga de variables de entorno
    ├── graphql/
    │   └── resolvers/ ← resolvers de Apollo por entidad
    ├── models/       ← capa de acceso a datos (driver nativo Mongo)
    ├── middleware/    ← JWT y manejo de errores
    ├── utils/         ← validadores y errores personalizados
    └── seed/          ← carga de datos iniciales
```

¿Qué es un .gitkeep?

Git no versiona carpetas vacías, solo ficheros. Para que la estructura de carpetas sobreviva al clonado inicial aunque todavía no haya código dentro, colocamos un fichero vacío llamado `.gitkeep` en cada subcarpeta del árbol. Es una convención ampliamente aceptada en la comunidad.

4.2. Archivos de configuración

4.2.1. package.json

Es el fichero central de todo proyecto Node. Describe el proyecto, sus dependencias y los scripts de ejecución. En el caso del Producto 3, el equipo ha configurado lo siguiente:

```
{
  "name": "fullstackattack-producto3",
  "version": "1.0.0",
  "description": "Producto 3 FP.450 – Backend GraphQL + MongoDB (JobConnect)",
  "type": "module",
  "main": "src/index.js",
  "scripts": {
    "start": "node src/index.js",
    "dev": "nodemon src/index.js",
    "seed": "node src/seed/ejecutarSeed.js",
    "db:up": "docker compose up -d",
    "db:down": "docker compose down",
    "db:logs": "docker compose logs -f mongo"
  },
  "engines": { "node": ">=20.0.0" }
}
```

Puntos relevantes:

Campo	Valor	Significado
"type"	"module"	Habilita la sintaxis ES Modules (import/export), la misma usada en Producto 2.
"main"	src/index.js	Fichero de entrada que arranca el servidor Express + Apollo.
"start"	node src/index.js	Lanza el servidor en modo producción.
"dev"	nodemon src/index.js	Lanza el servidor con recarga automática al guardar cambios. Es el comando del día a día.
"seed"	node src/seed/...	Ejecuta el script que siembra MongoDB con usuarios y publicaciones iniciales.
"db:up"	docker compose up -d	Levanta los contenedores de MongoDB y Mongo Express en segundo plano.
"db:down"	docker compose down	Apaga los contenedores sin borrar los datos.
"db:logs"	docker compose logs -f mongo	Muestra los logs de MongoDB en tiempo real (útil para depurar conexiones).
"engines"	node >= 20	Declara el requisito mínimo de Node. Ayuda a detectar temprano si un compañero clona con una versión antigua.

4.2.2. .gitignore

Indica a Git qué ficheros y carpetas nunca deben versionarse. El enunciado exige expresamente excluir node_modules. Se añaden otras exclusiones habituales para mantener el repositorio limpio:

- node_modules/ — se regenera con npm install, no tiene sentido subirlo.
- .env — contiene secretos (JWT secret, URI de Mongo en Atlas, contraseñas).
- logs y archivos temporales del sistema operativo.
- Configuración personal de IDEs (.idea/, .vscode/).
- Datos locales de los volúmenes de MongoDB (mongo-data/).

⚠ ¿Por qué .env NUNCA se sube?

El archivo .env contiene el JWT_SECRET con el que firmamos los tokens de autenticación. Si alguien lo obtiene, puede generar tokens válidos y suplantar al administrador. Los bots que rastrean GitHub

buscando secretos filtrados son una amenaza real: cualquier credencial subida por accidente se encuentra en minutos.

4.2.3. .env.example y .env

Los proyectos profesionales separan la configuración del código. Las variables que cambian entre entornos (desarrollo, testing, producción) se leen desde variables de entorno. En Node, dotenv carga automáticamente un fichero .env al arrancar.

El equipo mantiene dos ficheros:

Fichero	Se sube a Git	Contenido
.env.example	Sí	Plantilla con nombres de variables y valores genéricos. Sirve como guía para que cualquier miembro sepa qué rellenar al clonar.
.env	No	Fichero real con valores particulares de la máquina: URI de la BBDD, JWT secret único, credenciales del admin.

Las variables definidas son las siguientes:

```
# --- Servidor Express / Apollo ---
PORT=4000
NODE_ENV=development

# --- MongoDB ---
MONGO_URI=mongodb://admin:admin@localhost:27017
MONGO_DB_NAME=jobconnect_producto3

# --- Autenticación admin (JWT) ---
JWT_SECRET=<cadena_aleatoria_larga>
JWT_EXPIRES_IN=12h

# --- Credenciales del administrador por defecto ---
ADMIN_EMAIL=admin@jobconnect.com
ADMIN_PASSWORD=admin1234
```

4.2.4. docker-compose.yml

Describe la infraestructura que levantamos con Docker. El equipo corre dos servicios en paralelo:

- **mongo:** contenedor con MongoDB 7, publicado en el puerto 27017 del host. Los datos se persisten en un volumen Docker llamado mongo-data, lo que significa que aunque el contenedor se destruya, los datos sobreviven.
- **mongo-express:** interfaz web de administración de Mongo, accesible en <http://localhost:8083>. Permite explorar colecciones, documentos e índices sin usar comandos. Arranca solo cuando el

contenedor de MongoDB está marcado como "healthy", gracias a un healthcheck que hace ping a la base de datos cada 10 segundos.

Puertos utilizados

MongoDB → puerto 27017 (estándar).

Mongo Express → puerto 8083 (internamente el contenedor escucha en 8081, pero lo remapeamos a 8083 para evitar conflictos con otras instalaciones de phpMyAdmin o WordPress que algún miembro del equipo pueda tener corriendo en paralelo).

4.3. Estructura de src/

La carpeta src/ contiene todo el código fuente del backend. Cada subcarpeta tiene una responsabilidad única y bien delimitada. Esta separación por capas facilita la mantenibilidad, el testing y la lectura, y es una práctica directamente valorada por la rúbrica en el criterio "código modular".

Carpeta	Responsabilidad
src/config	Conexión a MongoDB con el driver nativo y carga validada de variables de entorno.
src/graphql	Definición del schema GraphQL (typeDefs) con los Types, Query y Mutation.
src/graphql/resolvers	Un archivo por entidad (usuarioResolver.js, publicacionResolver.js, seleccionadaResolver.js) con la lógica que ejecuta cada Query y Mutation.
src/models	Capa de acceso a datos. Encapsula las operaciones sobre las colecciones de MongoDB. Aquí vive el equivalente al almacenaje.js del Producto 2, pero usando el driver oficial.
src/middleware	Funciones que interceptan peticiones: verificación de JWT, manejo centralizado de errores.
src/utils	Utilidades puras: validadores (email, fecha, etc.) y clases de error personalizadas.
src/seed	Script para sembrar la base de datos con los usuarios y publicaciones iniciales del Producto 2.

5. Puesta en marcha en local

Este capítulo describe los pasos exactos que cada miembro del equipo debe seguir, una vez clonado el repositorio, para levantar el entorno de trabajo en su propia máquina. Todos los comandos se ejecutan desde la terminal integrada de VS Code (abierta con el atajo Ctrl+Ñ en Windows o Ctrl+º en Mac), posicionada dentro de la carpeta del proyecto.

5.1. Instalar dependencias del proyecto

Ejecutar una única vez tras clonar:

```
npm install
```

npm lee el fichero package.json y descarga todas las dependencias declaradas en node_modules/. La primera vez tarda entre 30 segundos y 2 minutos según la velocidad de red. Puede mostrar warnings de deprecación en dependencias anidadas: son normales e inofensivos mientras no aparezca ningún ERR!.

Dependencias instaladas

express — servidor web base.

@apollo/server + @as-integrations/express5 — Apollo Server v5 integrado con Express v5.

graphql — motor GraphQL.

mongoose — driver nativo (no Mongoose, como exige el enunciado).

jsonwebtoken + bcryptjs — autenticación JWT con contraseñas hasheadas.

dotenv — carga de .env.

cors — permisos CORS.

nodemon (dev) — recarga automática del servidor al guardar cambios.

5.2. Configurar el fichero .env

Duplicar el .env.example como .env. Este paso es distinto en Windows y macOS:

Windows (PowerShell)

```
Copy-Item .env.example .env
```

macOS (Terminal)

```
cp .env.example .env
```

Tras duplicarlo, abrir .env en VS Code y comprobar los valores. Para el desarrollo en local con Docker, los valores por defecto son correctos excepto el JWT_SECRET, que debe generarse de forma única en cada máquina.

5.3. Generar un JWT_SECRET único

El JWT secret es la clave criptográfica con la que el servidor firma los tokens de autenticación. Debe ser una cadena larga y aleatoria. La generamos con una orden de una sola línea aprovechando el módulo crypto incluido en Node:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

Ejecutar en la terminal copia y pega la salida, que será una cadena hexadecimal de 96 caracteres. Reemplazar el valor actual de JWT_SECRET en el fichero .env por esa cadena. La línea debe quedar con este aspecto (cadena distinta en cada máquina):

```
JWT_SECRET=c25da9358c8b4afda384459a6668348a0243d9126622304a59990a1e4486d6f37da5cbe8ef5f5ae06edec19650d4fce1
```

Cada máquina, su propio secret

Cada miembro del equipo debe generar su propio JWT_SECRET. No hay ninguna necesidad de compartirlo entre compañeros, ya que durante el desarrollo cada quien trabaja contra su propia base de datos local. Solo importa que el secret esté fijo dentro de cada máquina para que los tokens emitidos por el servidor sean verificables por el mismo servidor.

5.4. Levantar MongoDB con Docker Compose

Con Docker Desktop corriendo en segundo plano, ejecutar:

```
npm run db:up
```

Internamente esto invoca "docker compose up -d", que descarga (la primera vez) las imágenes de MongoDB 7 y Mongo Express 1.0.2, crea una red Docker privada entre ambos contenedores, y los arranca en segundo plano.

La primera ejecución tarda entre 1 y 3 minutos porque baja unos 700 MB de imágenes. A partir de la segunda vez el arranque es prácticamente instantáneo (menos de 5 segundos).

Verificar que los dos contenedores están vivos:

```
docker ps --filter "name=p3-"
```

La salida debe mostrar dos líneas con los contenedores p3-mongo (estado "healthy") y p3-mongo-express (estado "Up"). Si alguno aparece como "Exited" o no aparece, revisar los logs:

```
npm run db:logs
```

5.5. Acceder al panel Mongo Express

Mongo Express ofrece una interfaz web para inspeccionar la base de datos sin necesidad de comandos. Acceder desde el navegador a:

```
http://localhost:8083
```

El navegador pedirá un usuario y contraseña básicos. Usar:

Campo	Valor
Usuario	admin
Contraseña	admin

La pantalla principal muestra tres bases de datos preexistentes (admin, config, local), que son internas de MongoDB y no deben tocarse. La base jobconnect_producto3 no aparecerá hasta que el primer documento sea insertado desde el backend, ya que MongoDB crea las bases de datos de forma perezosa.

5.6. Comandos útiles del día a día

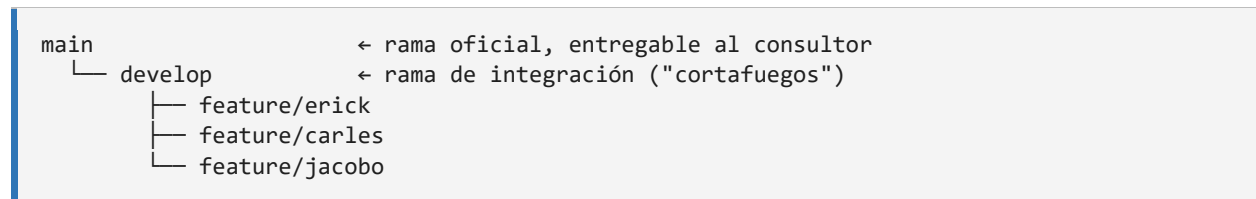
Comando	Efecto
npm run dev	Arrancar el servidor GraphQL con recarga automática. Uso habitual durante el desarrollo.
npm start	Arrancar el servidor en modo producción (sin nodemon).
npm run seed	Sembrar MongoDB con los usuarios y publicaciones iniciales del Producto 2.
npm run db:up	Levantar MongoDB y Mongo Express.
npm run db:down	Apagar los contenedores (sin borrar datos).
npm run db:logs	Ver los logs de MongoDB en tiempo real.
docker ps	Listar todos los contenedores en marcha.

6. Flujo de trabajo en equipo (GitFlow)

Con tres personas escribiendo código simultáneamente sobre un mismo repositorio, hace falta un protocolo que garantice que el trabajo de un miembro no pisa el de otro, que los cambios rotos nunca alcanzan la rama principal y que la entrega al consultor refleja siempre una versión estable. Para ello el equipo adopta una variante simplificada de GitFlow.

6.1. Estrategia de ramas

El repositorio tiene cinco ramas con roles claramente diferenciados:

		
Rama	Rol	¿Quién pushea directamente?
main	Versión oficial. Solo se actualiza desde develop al cerrar hitos.	Nadie. Solo vía Pull Request.
develop	Integración continua. Recoge el trabajo validado de todos los miembros.	Nadie. Solo vía Pull Request.
feature/erick	Trabajo personal de Erick.	Erick (push libre).
feature/carles	Trabajo personal de Carles Miguel.	Carles (push libre).
feature/jacobo	Trabajo personal de Jacobo.	Jacobo (push libre).

6.2. Reglas de oro del equipo

1. NADIE toca main directamente. main se actualiza exclusivamente mediante Pull Request desde develop, y solo cuando el equipo decide que el producto está listo para ser entregado o para cerrar una versión.
2. NADIE toca develop directamente. Los cambios entran en develop únicamente mediante Pull Request desde una rama feature/*, idealmente con revisión de al menos otro compañero.
3. Cada miembro trabaja exclusivamente en su propia rama feature/*. Si comete un error, lo arregla allí sin perjudicar al resto.
4. Antes de empezar la jornada, actualizar la rama personal con los cambios de develop para evitar conflictos acumulados.

5. Al finalizar una funcionalidad, abrir Pull Request hacia develop, describiendo qué hace el cambio y cómo probarlo.

6.3. Flujo diario recomendado

Al empezar a trabajar cada día, el primer gesto siempre es actualizar la rama personal:

```
# 1. Traer los últimos cambios del remoto
git checkout develop
git pull origin develop

# 2. Volver a la rama personal
git checkout feature/erick      # sustituir por la propia

# 3. Incorporar los cambios de develop a la rama personal
git merge develop

# 4. Si hubo conflictos, resolverlos en VS Code, añadir y commitear
# 5. Empezar a programar
```

Durante el desarrollo, commitear frecuentemente con mensajes claros:

```
git add <archivos_modificados>
git commit -m "feat(publicacion): add createPublicacion mutation"
git push
```

6.4. Convenciones de commits (Conventional Commits)

Todos los mensajes de commit del proyecto siguen el estándar Conventional Commits. Este formato facilita entender el historial de un vistazo, permite generar changelogs automáticamente, y es una buena práctica universal en proyectos profesionales. Puntúa positivamente en la rúbrica como evidencia de organización.

Estructura:

```
<tipo>(<ámbito>): <descripción corta en imperativo>

[cuerpo opcional con detalles]
```

Tipos utilizados en este proyecto:

Tipo	Cuándo usarlo	Ejemplo
feat	Nueva funcionalidad	feat(usuario): add createUsuario mutation
fix	Arreglar un bug	fix(auth): handle empty token header
chore	Tareas de proyecto (config, etc.)	chore: initialize Node.js project
build	Dependencias, Docker, build system	build(docker): add MongoDB via docker-compose
docs	Documentación	docs(readme): add setup instructions
refactor	Reestructurar sin cambiar comportamiento	refactor(models): extract validator helpers
test	Añadir o corregir tests	test(publicacion): add list pagination test

6.5. Apertura de Pull Requests

Una vez que una funcionalidad está terminada y probada en la rama personal, se abre un Pull Request hacia develop mediante la interfaz web de GitHub:

1. Acceder a <https://github.com/ErickColl/FullStackAttack-Producto3/pulls>.
2. Pulsar "New pull request".
3. Seleccionar base: develop y compare: feature/<alias>.
4. Pulsar "Create pull request". Añadir un título descriptivo (mismo estilo que los commits) y un cuerpo que explique los cambios, cómo probarlos y qué criterios de la rúbrica cubre.
5. Pedir revisión a al menos un compañero mediante la casilla "Reviewers".
6. Solo tras recibir aprobación, pulsar "Merge pull request".
7. Borrar la rama temporal si corresponde, o dejarla para seguir trabajando.

Protección de ramas (opcional pero muy recomendable)

Desde la configuración del repositorio en GitHub (Settings → Branches) se pueden activar "branch protection rules" sobre main y develop que impidan físicamente los pushes directos y exijan Pull Request con aprobación. Activarlo eleva el rigor del flujo y es un indicador profesional que el consultor valora.

7. Estado actual del proyecto

Este capítulo recoge una foto fija del avance del Producto 3 en el momento de redacción de esta guía. Permite al lector saber qué está operativo, qué queda por implementar y qué criterios de la rúbrica se cumplirán gracias a cada entrega parcial. El proyecto ha avanzado rápido: tiene cinco fases cerradas, cuatro Pull Requests mergeados a develop, y el backend ya persiste en MongoDB con datos iniciales, índices y validaciones completas. Sólo quedan por delante la autenticación JWT y los entregables finales.

7.1. Fase 1 — Infraestructura (COMPLETADA)

Todo lo descrito en los capítulos 3, 4 y 5 está terminado y commiteado al repositorio remoto. Incluye:

- Repositorio GitHub nuevo e independiente del Producto 2.
- Flujo GitFlow con ramas main, develop y feature/* operativas.
- package.json configurado como ES Module con scripts y engines.
- .gitignore validado excluyendo node_modules, .env, logs y las subcarpetas internas que genera la aplicación Postman.
- .env.example versionado y .env local generado con JWT_SECRET único.
- docker-compose.yml que levanta MongoDB 7 y Mongo Express 1.0.2.
- Dependencias de producción y desarrollo instaladas (express, @apollo/server, @as-integrations/express5, graphql, mongodb, jsonwebtoken, bcryptjs, dotenv, cors, nodemon).
- Estructura de carpetas src/ creada con .gitkeep para preservar el árbol vacío.
- Cuatro commits atómicos siguiendo Conventional Commits en main, más un quinto en develop con la estructura de carpetas.

7.2. Fase 2 — Backend inicial con Express, Apollo y MongoDB (COMPLETADA)

Esta fase la implementó el compañero Carles en la rama feature/carles y se integró en develop mediante el Pull Request #3 tras ser revisada por Erick. El objetivo era poner en pie el esqueleto funcional del backend: un servidor Express que expone un endpoint GraphQL alimentado por Apollo Server, con conexión real a la base de datos MongoDB del contenedor Docker. Al terminar esta fase, el servidor ya arrancaba, conectaba a Mongo y respondía a las primeras queries de prueba (saludo, estado) desde Apollo Sandbox.

- Servidor Express 5 con middlewares CORS y express.json.
- Apollo Server 5 integrado mediante @as-integrations/express5.
- Conexión a MongoDB vía driver nativo oficial con patrón singleton (connectToMongo + getDb).
- Schema GraphQL mínimo con queries de salud (saludo, estado).

- Endpoint operativo en <http://localhost:4000/graphql>, probado end-to-end en Apollo Sandbox.
- Revisión cruzada documentada en el PR #3 con veredicto de aprobación y lista de mejoras pendientes.

7.3. Fase 2.5 — Refuerzo y profesionalización del backend (COMPLETADA)

Tras aceptar el PR #3 de Carles, Erick implementó en la rama `feature/erick` un conjunto de mejoras transversales sobre la base recién creada, que se integraron en `develop` mediante el Pull Request #4. El objetivo era elevar la calidad del código al nivel que exige la rúbrica para la nota máxima en los criterios de código modular y seguro, óptimo manejo de errores y documentación con JSDoc bien detallada.

- Módulo `src/config/env.js` como única fuente de verdad para las variables de entorno, con validación obligatoria al arrancar y objeto inmutable congelado con `Object.freeze`.
- Refactor de `src/config/db.js`: eliminación de la llamada duplicada a `dotenv.config`, opciones de `timeout` explícitas en `MongoClient` (`serverSelectionTimeoutMS`, `connectTimeoutMS`) para evitar cuelgues cuando Mongo no está disponible, y nueva función `closeMongo` para cierre limpio.
- Middleware `src/middleware/errorHandler.js` con dos handlers globales: `notFoundHandler` (devuelve JSON 404 estructurado para rutas desconocidas) y `errorHandler` (respuesta JSON consistente, oculta el stack trace en producción).
- Refactor de `src/index.js` con graceful shutdown que atrapa `SIGINT` y `SIGTERM` y cierra Apollo y MongoDB ordenadamente antes de terminar el proceso.
- `process.exit(1)` si falla el arranque, para que Docker, PM2 o cualquier orquestador detecten el fallo.
- Apollo introspection activada sólo en desarrollo (`env.isDevelopment`).
- Endpoint raíz / transformado en JSON informativo (`service`, `status`, `graphql`, `environment`) para health checks externos.
- JSDoc completa con `@param`, `@returns`, `@throws` y `@typedef` en todas las funciones nuevas y modificadas.

7.4. Fase 3 — CRUD en memoria de las tres entidades (COMPLETADA)

Esta fase la implementó Erick en la rama `feature/erick` y se integró en `develop` mediante el Pull Request #6. Porta al backend la lógica de persistencia que en el Producto 2 vivía en el fichero `almacenaje.js` del navegador, pero contra arrays en memoria en lugar de MongoDB. Trabajar primero en memoria permitió iterar rápido sobre las validaciones, las normalizaciones y la coordinación entre entidades, sin arrastrar en paralelo la complejidad de la migración a Mongo. Esa migración fue la siguiente fase y el código quedó preparado para que sólo cambiaran los ficheros de la carpeta `models`, sin tocar resolvers, schema, validadores ni errores.

El resultado son 10 queries y 7 mutations definidas, documentadas con triple-quoted descriptions y probadas end-to-end en Apollo Sandbox. La arquitectura adoptada es `thin resolvers`, `fat models`: los

resolvers sólo adaptan argumentos y delegan, mientras que toda la lógica de negocio (normalización, validación, persistencia) vive en los models. Los errores están tipados (ValidationError, NotFoundError, ConflictError, UnauthorizedError, ForbiddenError) y el middleware de errores ya existente los transforma automáticamente en respuestas HTTP con el status code correcto.

- Clases de error personalizadas en `src/utils/errors.js` con `statusCode` y `code`, consumidas por el `errorHandler` existente.
- Validadores reutilizables en `src/utils/validators.js` portados del Producto 2, incluyendo un fix del bug del overflow silencioso de fechas: el P2 aceptaba '2026-02-30' porque JavaScript lo autocorregía a marzo. Ahora el backend rechaza fechas imposibles reconstruyendo y comparando año, mes y día exactos.
- Entidad Usuario con model in-memory (sembrado con Laura, Carlos y Ana como en el P2), `serializarUsuario` que garantiza que el password nunca sale del model, y rol ampliado con admin para la autenticación JWT de la Fase 5.
- Entidad Publicación con model in-memory (sembrado con las 4 publicaciones iniciales del P2), listado ordenado por fecha descendente, filtro por tipo (oferta/demanda) y recuento agregado para el dashboard.
- Entidad Seleccionada que rastrea qué publicaciones están en el panel del dashboard. Idempotente al añadir y con limpieza automática de selecciones huérfanas cuando se elimina una publicación.
- Coordinación inter-entidades en el resolver de `eliminarPublicación`: invoca `limpiarSeleccionesHuérfanas` para mantener el panel coherente atómicamente.
- Query `resumenDashboard` que devuelve el payload exacto de las cajitas de totales del P2: `totalOfertas`, `totalDemandas`, `totalUsuarios` y `totalSeleccionadas`.
- Seis commits atómicos siguiendo Conventional Commits, cada uno con una unidad lógica de cambio (errores, validadores, usuario, publicación, seleccionada y wiring del schema).

7.5. Fase 4 — Persistencia real en MongoDB (COMPLETADA)

Esta fase la implementó el compañero Carles en la rama `feature/carles` y se integró en `develop` mediante dos Pull Requests consecutivos: el PR #7 (migración de Usuario y Publicación, más adaptación asíncrona de Seleccionada) y el PR #8 (migración completa de Seleccionada, seed inicial e índices). Ambos fueron revisados por Erick antes del merge y quedaron documentados con veredicto de aprobación en los comentarios del PR. El objetivo era sustituir los arrays en memoria de la Fase 3 por colecciones reales de MongoDB usando el driver nativo, sin romper nada del trabajo anterior y respetando al 100 % la interfaz pública de los models.

La decisión clave que hizo posible esta migración sin efectos secundarios fue haber mantenido la arquitectura `thin resolvers / fat models` desde la Fase 3. Toda la sustitución de la capa de persistencia se concentró en los tres ficheros de `src/models/`, y ni los resolvers ni el schema GraphQL ni los validadores

ni las clases de error necesitaron el más mínimo cambio. Este nivel de aislamiento entre capas es exactamente lo que la rúbrica valora bajo los criterios de código modular y correcto uso de patrones.

Desde el punto de vista operativo, el sistema pasa de ser una simulación a comportarse como un backend de producción: los datos sobreviven a los reinicios del servidor, varias instancias podrían compartir una misma base de datos y las consultas se apoyan en índices de MongoDB para ser eficientes. A nivel de código, el cambio más notable es que todas las funciones de los models pasan a ser asíncronas (async/await), puesto que el driver de MongoDB devuelve siempre Promises.

- Entidad Usuario migrada a la colección usuarios de MongoDB. Las operaciones listarUsuarios, buscarUsuarioPorId, buscarUsuarioPorEmail, crearUsuario, eliminarUsuarioPorEmail y loguearUsuario ahora consultan la base de datos. Se preserva serializarUsuario para garantizar que el campo password nunca sale del model en ninguna respuesta GraphQL.
- Entidad Publicación migrada a la colección publicaciones. Las operaciones listarPublicaciones, buscarPublicacionPorId, listarPublicacionesPorTipo, crearPublicacion, eliminarPublicacionPorId y contarPublicaciones ahora leen y escriben en Mongo. La ordenación por fecha descendente y los filtros por tipo se mantienen idénticos a los de Fase 3, respetando el comportamiento externo visible.
- Entidad Seleccionada migrada a la colección seleccionadas (PR #8). El array const seleccionadas = [] ha desaparecido por completo. Las selecciones ahora persisten en MongoDB, sobreviven a reinicios del servidor y se benefician de un índice único en el campo publicacionId para prevenir duplicados aunque lleguen peticiones concurrentes.
- Script de seed en src/seed/seed.js que inserta los mismos 3 usuarios y 4 publicaciones del Producto 2 cuando la base de datos está vacía. Es idempotente: comprueba countDocuments antes de insertar, por lo que ejecutarlo múltiples veces no duplica datos. El seed se invoca automáticamente al arrancar el servidor, tras conectar a Mongo y antes de levantar Apollo.
- Creación automática de seis índices al arrancar: email único en usuarios (blinda la regla de negocio a nivel de base de datos, no sólo en el JavaScript), id único en usuarios y publicaciones, índices no únicos de tipo y fecha en publicaciones para acelerar los filtros habituales del dashboard, y publicacionId único en seleccionadas para garantizar que una publicación no pueda ser seleccionada dos veces.
- Campo id numérico preservado como identificador funcional de la API (además del _id interno que genera MongoDB). Esto mantiene la compatibilidad con los resolvers, el schema GraphQL y la lógica heredada, evitando que un cambio de tipo de dato se propague a todas las capas superiores.
- Integración del seed en src/index.js respetando toda la infraestructura de la Fase 2.5: el graceful shutdown para SIGINT y SIGTERM, el closeMongo, los middlewares de errores y la introspección condicional permanecen intactos. El seed se ejecuta como un paso más dentro del flujo de arranque, no como un script suelto.
- Tres commits atómicos en el PR #8 (feat(seleccionada) + feat(seed) + Update index.js) y dos commits en el PR #7, todos con Conventional Commits y con diffs acotados al ámbito declarado.

Durante la revisión del PR #8 se detectaron dos puntos menores que no bloquean el merge y que quedan explícitamente diferidos a la Fase 5, puesto que caen en su ámbito de autenticación: las contraseñas del seed se insertan todavía en texto plano (deberán hashearse con bcryptjs para que el futuro loginAdmin pueda compararlas), y falta un usuario administrador predefinido (tipo admin@jobconnect.com con rol admin) necesario para que Jacobo pueda probar el login de administrador. Ambos puntos están documentados en el comentario de aprobación del PR #8 para que el responsable de la Fase 5 los tenga presentes.

7.6. Fase 5 — Autenticación JWT del administrador (COMPLETADA)

Esta fase la implementó el compañero Jacobo en la rama feature/jacobo y se integró en develop mediante el Pull Request #9. El objetivo era añadir una capa de seguridad real al backend del Producto 3: hasta este punto, cualquier petición que llegara al endpoint GraphQL podía ejecutar operaciones tan sensibles como crear o eliminar usuarios y publicaciones. La rúbrica del producto exige protección por JWT, y este PR cierra ese requisito con un patrón estándar de la industria.

La idea principal es que el administrador inicia sesión, recibe un token JWT firmado por el servidor y lo envía en el header Authorization de las peticiones siguientes. El backend verifica el token en cada request y decide si la operación se puede ejecutar o no. Las queries de lectura se mantienen públicas porque el dashboard del frontend del Producto 2 las necesita sin autenticación; las mutations sensibles, en cambio, quedan protegidas y exigen rol admin.

El cambio se reparte entre 8 ficheros, con 3 commits atómicos en el orden correcto: primero los helpers de contexto (la capa base), después la mutation de login (que genera el token) y por último la protección de las mutations (que consume el token). Es un orden ejemplar: cada commit deja el sistema en un estado funcional aunque no se aplique el siguiente.

- Nuevo módulo src/middleware/auth.js (113 líneas) con tres responsabilidades aisladas: extraerTokenDesdeRequest parsea el header Authorization, obtenerUsuarioDesdeRequest verifica el token con jwt.verify y devuelve null si falla (no lanza, para no bloquear queries públicas), y requireAuth + requireAdmin lanzan UnauthorizedError o ForbiddenError respectivamente desde los resolvers protegidos.
- Distinción clara entre 401 UnauthorizedError (sin token o token inválido) y 403 ForbiddenError (token válido pero el usuario no es admin). Esta diferencia importa cuando un frontend tiene que decidir si pedir login o mostrar un mensaje de permisos insuficientes.
- Nuevo tipo GraphQL AuthPayload { token: String!, usuario: Usuario! } y nueva mutation loginAdmin(email, password): AuthPayload! añadidos a src/graphql/typeDefs.js, con descripciones y errores documentados con triple-quoted strings.
- Función loginAdmin en src/models/usuarioModel.js que normaliza el email y la password, valida formato, busca el usuario por email, comprueba que tenga rol admin y compara la password contra el hash bcrypt almacenado. Devuelve siempre UnauthorizedError ante cualquier fallo, sin

distinguir si el problema es que el email no existe, que el rol no es admin o que la password es incorrecta. Esta uniformidad es defensa estándar contra enumeración de usuarios.

- Función `generarTokenAdmin` en `src/graphql/resolvers/usuarioResolver.js` que firma el JWT con `env.jwtSecret`. El payload incluye solo email, rol y subject (id del usuario, usando el campo estándar sub del estándar JWT), nunca la contraseña ni datos sensibles. La expiración por defecto es 12 horas, configurable en env.
- Modificación mínima en `src/index.js` para enriquecer el contexto de Apollo: `context: async ({ req }) => ({ usuario: obtenerUsuarioDesdeRequest(req) })`. Cualquier resolver puede ahora consultar `context.usuario` para saber si la petición está autenticada.
- Mutations protegidas con `requireAdmin(context)`: `crearUsuario`, `eliminarUsuario`, `crearPublicacion`, `eliminarPublicacion`, `anadirSeleccionada` y `quitarSeleccionada`. La mutation `loginAdmin` queda deliberadamente pública porque es el punto de entrada para conseguir el token. La mutation legacy `loguearUsuario` también se mantiene pública para no romper compatibilidad con el trabajo previo.
- Función `asegurarAdminInicial` en `src/seed/seed.js` que garantiza la existencia del usuario `admin@jobconnect.com` con rol admin y password hasheada con `bcrypt` (10 salt rounds). Implementada con patrón `upsert` manual: si el admin existe lo actualiza, si no lo inserta. Se ejecuta tanto si la base de datos está vacía como si ya tenía datos del Producto 2, cubriendo el punto que quedó pendiente del PR #8.
- Cuatro nuevas variables en `src/config/env.js`: `JWT_SECRET` y `ADMIN_EMAIL/ADMIN_PASSWORD` obligatorias (sin ellas el servidor no arranca), `JWT_EXPIRES_IN` opcional con default 12h. El fichero `.env.example` se actualizó con comentarios claros que explican el formato y avisan de que en producción `JWT_SECRET` debe ser una cadena aleatoria de al menos 32 caracteres.

La fase se probó end-to-end en Apollo Sandbox confirmando los cuatro comportamientos esperados: `loginAdmin` con credenciales correctas devuelve un token JWT firmado, `loginAdmin` con password incorrecta devuelve `UnauthorizedError`, una mutation protegida sin token devuelve `UnauthorizedError`, y la misma mutation con el token enviado como Bearer Token se ejecuta correctamente. Estos cuatro casos quedaron formalizados como tests automáticos en la colección Postman que se construyó en la siguiente fase.

La review del PR #9 documentó tres observaciones menores que no bloquean el merge: la cabecera de `usuarioModel.js` todavía dice estado actual (Fase 4), la JSDoc de `loguearUsuario` menciona una migración a `bcrypt` que se decidió no hacer por compatibilidad, y la mutation `crearUsuario` no hashea la password al insertar. Este último punto es un bug latente: si en algún momento se crea un segundo administrador a través de la API (con `requireAdmin` permitido), su password quedaría en texto plano y `loginAdmin` no podría compararla con `bcrypt`. Como el seed ya garantiza un admin hasheado y la API no se usa para crear más admins, no es bloqueante; queda como deuda técnica menor para futuras iteraciones.

7.7. Fix del enmascaramiento de errores en Apollo (PR #10)

Durante la creación de la colección Postman para la Fase 6, los tests negativos de la nueva carpeta 5. Tests de seguridad detectaron un comportamiento incorrecto en el backend: todos los errores tipados de la aplicación (`UnauthorizedError`, `ValidationError`, `NotFoundError`, `ConflictError`, `ForbiddenError`) llegaban al cliente con `extensions.code = INTERNAL_SERVER_ERROR` en lugar de su código semántico real. Y peor todavía: Apollo incluía el `stacktrace` completo en la respuesta JSON, exponiendo rutas absolutas del sistema de ficheros del servidor y la lista de funciones internas al cliente.

La causa raíz era una decisión heredada de la Fase 3: la clase base `AppError` de `src/utils/errors.js` extendía de `Error` plano, no de `GraphQLError`. Apollo Server reserva un trato especial para los errores que extienden de su propia clase `GraphQLError`: respeta el campo `extensions.code` y no incluye el `stacktrace` en la respuesta. Para cualquier otra excepción, asume que es un crash imprevisto, enmascara el código como `INTERNAL_SERVER_ERROR` para no revelar detalles del fallo y, paradójicamente, sí incluye el `stacktrace` en modo desarrollo.

El arreglo es quirúrgico: hacer que `AppError` extienda de `GraphQLError` (en lugar de `Error`) y, en su constructor, llamar a `super(message, { extensions: { code } })`. Apollo entonces reconoce el error como esperado y respeta el código tipado. Las propiedades sueltas `statusCode` y `code` se preservan en cada instancia para que el `errorHandler middleware` de Express (que escribió Carles y mantenemos desde la Fase 2.5) siga funcionando sin cambios. Las subclases (`ValidationError`, `UnauthorizedError`, `ForbiddenError`, `NotFoundError`, `ConflictError`) mantienen su firma pública intacta: mismos nombres, mismos constructores, mismos códigos.

- Un solo fichero modificado (`src/utils/errors.js`), 65 líneas añadidas y 61 eliminadas.
- Cero dependencias nuevas: el paquete `graphql` ya estaba en `package.json` desde la Fase 1, así que no se añade ni se actualiza nada.
- Cero cambios en otros ficheros del proyecto: la API pública de las clases no cambia, así que todos los `throw new ValidationError(...)`, `throw new NotFoundError(...)`, etc. esparcidos por `validators.js`, los `models`, `middleware/auth.js` y los `resolvers` funcionan igual sin tocarlos.
- Mejora real de seguridad: el cliente ya no recibe el `stacktrace`, así que un atacante deja de poder mapear la estructura interna del backend a partir de errores.
- Mejora real de UX para el frontend: el cliente puede ahora distinguir un error de validación (`VALIDATION_ERROR`), de auth (`UNAUTHORIZED`), de permisos (`FORBIDDEN`), de recurso no encontrado (`NOT_FOUND`) o de conflicto (`CONFLICT`) y reaccionar diferente para cada uno.

El bug se descubrió porque la carpeta 5 de la colección Postman incluye tests negativos con `assertions` explícitos del tipo `pm.expect(json.errors[0].extensions.code).to.eql("UNAUTHORIZED")`. Este es exactamente el tipo de hallazgo que justifica la inversión en tests negativos en una colección de testing profesional: en lugar de limitarse al camino feliz, se valida también el contrato del backend ante errores. La rúbrica del Producto 3 valora esta práctica como indicador de calidad del proceso, no solo del resultado.

7.8. Colección Postman y tests automatizados (PR #11)

Con el backend cerrado tras Fase 5 y el bug del enmascarado de errores ya arreglado, Erick construyó la colección Postman oficial del proyecto y la commiteó al repositorio en el PR #11. Esta colección es uno de los entregables formales de la Fase 6 y, junto con la URL pública desplegada, es la herramienta principal con la que el consultor podrá probar el backend sin tener que instalar nada localmente.

La colección cubre las 17 operaciones GraphQL del schema (10 queries y 7 mutations) más 2 tests negativos de seguridad. Está organizada en 6 carpetas numeradas con prefijos 0., 1., 2., etc., para que el Collection Runner las ejecute en un orden predecible y los tests no fallen por dependencias entre requests. La estructura final es: 0. Healthcheck (2 queries), 1. Auth (login admin), 2. Usuario (5 requests), 3. Publicación (6 requests), 4. Dashboard / Seleccionadas (6 requests), y 5. Tests de seguridad (2 requests).

- Token JWT gestionado automáticamente. La request 1. Auth → Login Admin incluye un script post-response que extrae el campo `data.loginAdmin.token` de la respuesta y lo guarda en la variable de entorno `{{token}}`. Todas las requests protegidas envían `Authorization: Bearer {{token}}` sin necesidad de copiar y pegar el token a mano. Es el patrón profesional de gestión de credenciales en colecciones Postman.
- Encadenamiento de requests. La request 3.5 Crear publicación guarda el id de la publicación recién creada en `{{ultimaPublicacionId}}`, y la request 3.6 Eliminar publicación consume esa variable. Esto permite ejecutar el ciclo crear → eliminar dentro del Collection Runner sin pegar ids manualmente.
- 60+ tests automáticos (assertions ejecutables) repartidos por las 22 requests. Validan status code 200, presencia de data o errors según corresponda, códigos en `extensions.code`, invariantes de dominio (ofertas + demandas === total en el recuento, ordenación por fecha descendente en el listado), y garantías de seguridad (el campo `password` nunca aparece en ningún tipo Usuario devuelto).
- Tests negativos en la carpeta 5: la 5.1 Mutación SIN token verifica que la respuesta tenga code `UNAUTHORIZED` y que la operación NO se haya ejecutado, y la 5.2 Login con password incorrecta verifica también `UNAUTHORIZED` y que el mensaje sea genérico (no revela si el problema es el email o el password) para evitar enumeración de usuarios.
- Environment JobConnect — Local con 4 variables: `baseUrl` (`http://localhost:4000/graphql`), `adminEmail` (`admin@jobconnect.com`), `adminPassword` (`admin1234`) y `token` (vacío al inicio, lo rellena el Login). Exportado como JSON al repositorio.
- README.md de la carpeta `postman/` con instrucciones de importación, activación del environment, ejecución del Collection Runner e interpretación de los resultados. Documenta también la estructura de carpetas y el flujo de uso recomendado.

El Collection Runner ejecuta las 22 requests en orden en menos de cinco segundos y muestra un informe con los 60+ tests pasando o fallando. Tras el merge del PR #10 (fix de errores), el resultado del Runner es 22/22 requests OK con todos los tests verdes, lo cual confirma end-to-end que el backend cumple su contrato en el camino feliz y rechaza correctamente los caminos negativos.

7.9. Despliegue: MongoDB Atlas y CodeSandbox (PR #12)

El último entregable técnico de la Fase 6 es el despliegue del backend en una plataforma cloud accesible públicamente. El equipo eligió la combinación estándar para proyectos Node.js + MongoDB sin infraestructura propia: MongoDB Atlas como base de datos cloud y CodeSandbox como runner del servidor. Ambas en plan free tier, sin tarjeta de crédito, sin caducidad.

MongoDB Atlas

Atlas es el servicio cloud oficial de MongoDB. La cuenta del proyecto se creó a nombre de Erick con la integración SSO de GitHub. El cluster es un M0 free tier (512 MB de almacenamiento, sin coste, sin caducidad), alojado en AWS Paris (eu-west-3) con MongoDB 8.0.21 en topología Replica Set de tres nodos, lo que da alta disponibilidad sin trabajo extra.

- Database User dedicado al backend: usuario jobconnect-app con rol Read and write to any database. Sigue el principio de mínimo privilegio (no es Atlas admin), pero permite leer, escribir y crear índices que es lo que necesita el seed al arrancar.
- Network Access configurado con dos entradas: la IP del PC de desarrollo (creada automáticamente por Atlas durante el setup inicial) y 0.0.0.0/0 para que CodeSandbox y otros entornos cloud puedan conectar. La protección efectiva la dan las credenciales del Database User, no la lista de IPs.
- URI de conexión con formato mongodb+srv://. La password del Database User contiene caracteres especiales que requieren URL-encoding (! como %21 y @ como %40) para que el parser de la URI no los interprete como separadores estructurales. Sin esta codificación, el cliente fallaría con auth error sin mostrar la causa real.
- URI almacenada exclusivamente en variables de entorno locales (.env del PC de desarrollo) y en los secretos cifrados de CodeSandbox. Nunca se commitea al repositorio. El .env.example sigue apuntando a Docker local para que cualquier miembro del equipo pueda reproducir el entorno sin Atlas.
- Misma base de datos jobconnect_producto3 que en Docker local: 3 colecciones (usuarios, publicaciones, seleccionadas), 6 índices, los 4 usuarios + 4 publicaciones del seed más el administrador con password hasheada con bcrypt.

CodeSandbox

CodeSandbox es un entorno de desarrollo en cloud que ejecuta proyectos Node.js dentro de microVMs y publica una URL pública persistente para cada uno. El plan Free incluye 400 créditos al mes (alrededor de 40 horas de VM ininterrumpida), suficiente para un proyecto académico que solo se usa puntualmente.

- Cuenta creada vía SSO con GitHub, lo que permite importar el repositorio FullStackAttack-Producto3 sin configuración adicional.

- Devbox importado desde la rama develop (no main, que está congelada hasta la entrega final). CodeSandbox detecta package.json automáticamente y ejecuta npm install + npm run dev al arrancar.
- Las 8 variables de entorno requeridas (PORT, NODE_ENV, MONGO_URI, MONGO_DB_NAME, JWT_SECRET, JWT_EXPIRES_IN, ADMIN_EMAIL, ADMIN_PASSWORD) están configuradas como secretos cifrados en Editor → Repository → Env Variables. CodeSandbox las inyecta en process.env al arrancar la VM, exactamente como dotenv haría con un fichero .env real.
- Base de datos: el backend desplegado apunta al mismo cluster de Atlas que el backend local. Esto garantiza que los datos sean consistentes entre los dos entornos y que cualquier prueba que el equipo o el consultor haga en Atlas se vea reflejada en ambos sitios.
- Hibernación automática: CodeSandbox duerme la VM tras unos minutos sin actividad. Cuando alguien (el consultor, un miembro del equipo) abre la URL pública, la VM despierta sola en 30-60 segundos sin intervención manual. Esto preserva créditos y mantiene la URL siempre disponible.
- CORS verificado: el backend acepta peticiones cross-origin desde cualquier origen (cors() sin opciones), lo cual permite que Postman y herramientas externas hagan fetch a la URL pública sin bloqueos del navegador.

Verificación end-to-end

Se ejecutó la batería de verificación completa para confirmar que toda la pila funciona como producto cloud:

- Endpoint raíz <https://2cw562-4000.csb.app/> devuelve el JSON de healthcheck con service, status, graphql y environment.
- Endpoint GraphQL <https://2cw562-4000.csb.app/graphql> carga Apollo Sandbox con el schema introspectado en vivo, accesible desde cualquier navegador del mundo.
- Query resumenDashboard contra la URL pública devuelve totalUsuarios=4, totalOfertas=2, totalDemandas=2 (los datos del seed sembrados originalmente desde el PC local en Atlas).
- Mutation loginAdmin contra la URL pública devuelve un token JWT firmado correctamente.
- Postman ejecutado contra la URL pública con un environment paralelo JobConnect — CodeSandbox: las 22 requests de la colección funcionan idénticas, con latencia algo mayor (1-3 segundos vs 30-50 ms en local) pero todos los tests siguen verdes.
- Fetch cross-origin desde la consola de un navegador en google.com a la URL pública devuelve correctamente la query, confirmando CORS.

El PR #12 incluyó dos cambios coherentes: la documentación del despliegue en postman/README.md (URL pública, instrucciones para apuntar Postman a la URL del sandbox, gestión de secretos en CodeSandbox) y un endurecido del .gitignore que sustituyó la lista explícita de patrones .env (.env, .env.local, .env*.local) por un patrón comodín más amplio (.env*) con una excepción explícita para .env.example. Esto evita que cualquier variante futura de fichero .env (por ejemplo .env.atlas, .env.production o .env.local-backup) pueda commitearse al repositorio por descuido.

7.10. Historial de Pull Requests del proyecto

La tabla siguiente recoge el historial completo de Pull Requests del repositorio hasta la fecha de redacción de este documento. Es el reflejo directo del flujo de trabajo del equipo y la prueba documental de la disciplina GitFlow descrita en el capítulo 6. Cualquier consultor puede verificar la coherencia abriendo la pestaña Pull requests del repositorio en GitHub.

PR	Descripción y estado
#1	feature/erick → develop. Ajuste del .gitignore para no subir carpetas internas de Postman. Mergeado. En retrospectiva, este PR fue prematuro (un solo commit de mantenimiento no merecía promoción a develop); el aprendizaje quedó registrado en la sección 7.11.
#2	develop → main. Mergeado. También prematuro: main quedó con código que no corresponde a un hito de entrega. Se decidió no revertir para no ensuciar más el historial y dejar el caso como ejemplo didáctico para el vídeo explicativo.
#3	feature/carles → develop. Fase 2: servidor Express + Apollo + conexión a MongoDB. Mergeado tras revisión de Erick con veredicto de aprobación.
#4	feature/erick → develop. Fase 2.5: módulo env, refactor de db.js, middleware de errores, graceful shutdown y JSDoc completo. Mergeado.
#5	feature/erick → main. Cerrado sin merge: se detectó a tiempo que apuntaba erróneamente a main en lugar de a develop. Queda como ejemplo del rigor que el equipo aplica al revisar sus propias acciones.
#6	feature/erick → develop. Fase 3 completa: CRUD en memoria de Usuario, Publicación y Seleccionada. Mergeado tras revisión de los compañeros.
#7	feature/carles → develop. Fase 4 parcial: migración de Usuario y Publicación a MongoDB, más adaptación asíncrona de Seleccionada manteniendo su interfaz pública. Mergeado como paso intermedio para desbloquear el trabajo del equipo mientras Carles terminaba la segunda mitad.
#8	feature/carles → develop. Fase 4 completa: migración de Seleccionada a MongoDB, script de seed idempotente e índices automáticos al arrancar. Mergeado tras revisión detallada de Erick documentada en los comentarios del propio PR.
#9	feature/jacobo → develop. Fase 5 completa: middleware auth.js con helpers JWT, mutation loginAdmin con generación de token, protección de las 6 mutations sensibles con requireAdmin, y actualización del seed para garantizar el admin inicial con password hasheada con bcrypt. Mergeado tras revisión de Erick documentada en los comentarios del PR.
#10	feature/erick → develop. Fix del enmascarado de errores en Apollo: AppError pasa a extender de GraphQLError para que extensions.code se propague correctamente al cliente y para que Apollo deje de incluir stacktraces en la respuesta. Bug detectado por los tests negativos de la futura colección Postman. Mergeado.

PR	Descripción y estado
#11	feature/erick → develop. Colección Postman oficial del proyecto: 22 requests organizadas en 6 carpetas, 60+ tests automáticos, environment local, README de uso. Token JWT gestionado automáticamente desde el script post-response del Login Admin. Mergeado.
#12	feature/erick → main (por error). Documentación del despliegue de CodeSandbox en postman/README.md y endurecido del .gitignore con el patrón .env*. Se abrió accidentalmente contra main en lugar de develop. El error se detectó tras el merge: en lugar de revertir, se alineó develop hacia main con un fast-forward limpio (sin force-push, sin commits de revert). Resultado: main y develop apuntan al mismo commit (764e478) y el modelo GitFlow queda restaurado. La nota está documentada como comentario del propio PR #12.

7.11. Acuerdo del equipo: main congelada hasta la entrega

Tras el incidente de los PRs #1 y #2, el equipo adoptó formalmente la siguiente regla, que queda registrada en este documento y que cualquier consultor que evalúe el proyecto encontrará coherentemente aplicada a partir de ese momento:

La rama main está congelada hasta el hito de entrega final del Producto 3. Entre ahora y ese momento, todo el trabajo entra en develop únicamente mediante Pull Request desde una rama feature/*, y desde develop nadie promociona nada a main sin el acuerdo explícito de los tres miembros del equipo.

Esta regla cubre el criterio de la rúbrica que valora la disciplina de ramas, y además refleja fielmente cómo se trabaja en equipos profesionales: main es la rama que representa el producto listo para entregar al cliente, no un almacén de cambios en curso. En un proyecto real con despliegue automático, cualquier merge a main dispara una puesta en producción, por lo que sólo se hace cuando el conjunto del equipo considera el producto suficientemente estable.

Durante el cierre de Fase 6 se produjo una nueva infracción accidental de esta regla: el PR #12 se abrió por error contra main en lugar de contra develop, y se mergeó sin que el equipo se diera cuenta. El error se detectó al revisar el historial inmediatamente después y se rectificó limpiamente alineando develop hacia main con un fast-forward (git merge origin/main desde develop, sin force-push, sin revert). Tras la corrección, develop y main vuelven a apuntar al mismo commit y el modelo GitFlow queda restaurado. La intención del equipo de hacer una promoción formal develop → main como acto final de entrega se mantiene intacta: el PR final cerrará el círculo cuando esté lista la documentación, el vídeo y el PDF de entrega.

7.12. Información de la entrega

Para facilitar al consultor la revisión del Producto 3, esta sección centraliza todos los enlaces y recursos finales del proyecto. Cualquiera de los enlaces siguientes es accesible públicamente sin instalación previa,

sin necesidad de clonar el repositorio y sin credenciales adicionales más allá de las del propio backend (admin@jobconnect.com / admin1234, documentadas en .env.example).

Recurso	Enlace o ubicación
Repositorio GitHub	https://github.com/ErickColl/FullStackAttack-Producto3
Backend desplegado (raíz)	https://2cw562-4000.csb.app
Endpoint GraphQL público (Apollo Sandbox)	https://2cw562-4000.csb.app/graphql
Colección Postman	postman/JobConnect-Producto3.postman_collection.json (en el repositorio)
Environment Postman local	postman/JobConnect-Local.postman_environment.json (en el repositorio)
README de la colección Postman	postman/README.md (en el repositorio)
Documento técnico (este PDF)	docs/Documento_tecnico_Producto3_FullStackAttack.docx (en el repositorio)
Vídeo explicativo de equipo	Enlace en el PDF de entrega final preparado por Jacobo
PDF de entrega oficial	Subido al campus virtual junto con esta guía

Cuando el consultor abra el endpoint GraphQL en el navegador, cargará Apollo Sandbox directamente y podrá ejecutar cualquier query o mutation del schema sin instalar Postman. Si el sandbox lleva varios minutos sin actividad puede tardar entre 30 y 60 segundos en despertar la primera vez (el plan free tier de CodeSandbox hiberna las VMs inactivas para ahorrar créditos); a partir de ahí responde con normalidad.

8. Troubleshooting — Problemas comunes

Este capítulo recoge los errores más frecuentes que pueden aparecer al poner en marcha el entorno, junto con la solución exacta para cada uno. Consultar antes de llamar a un compañero.

8.1. "Docker daemon is not running" al ejecutar docker ps

Docker Desktop no ha terminado de arrancar todavía, o está directamente cerrado.

8.2. Puerto 27017 ya en uso al levantar MongoDB

Alguna instalación previa de MongoDB o algún contenedor anterior está ocupando el puerto.

```
# Windows (PowerShell)
Get-Process | Where-Object { $_.Path -match "mongo" }

# macOS / Linux
lsof -i :27017
```

Si es otro contenedor Docker, bastar con "docker stop <nombre>". Si es una instalación nativa de MongoDB, parar el servicio desde el gestor de servicios del sistema.

8.3. npm install falla con errores de permisos en macOS

En algunas configuraciones de macOS, npm intenta escribir en una carpeta global protegida.

```
mkdir ~/.npm-global
npm config set prefix "~/.npm-global"
echo 'export PATH=~/.npm-global/bin:$PATH' >> ~/.zshrc
source ~/.zshrc
```

8.4. "fatal: not a git repository" al clonar

Normalmente ocurre cuando el punto al final del comando git clone se olvida, o cuando se ha clonado accidentalmente dentro de una subcarpeta.

8.5. VS Code no reconoce el comando "code" en la terminal

El ejecutable de VS Code no está en el PATH del sistema.

8.6. "Permission denied" al pushear a GitHub

Git no ha almacenado las credenciales o la cuenta no tiene permisos sobre el repositorio.

```
# Windows (PowerShell)
```



```
git config --global --unset credential.helper

# macOS
git credential-osxkeychain erase
host=github.com
protocol=https
```

Tras esto, el siguiente push pedirá volver a autenticar contra GitHub abriendo el navegador.

8.7. Conflictos al hacer merge de develop en la rama personal

Dos compañeros han tocado las mismas líneas en ramas distintas.

```
git add <ficheros_resueltos>
git commit -m "fix(merge): resolve conflicts with develop"
git push
```

8.8. MongoDB Express no carga en el navegador

El contenedor de Mongo Express depende del contenedor de Mongo. Si Mongo no arrancó correctamente, Mongo Express queda en bucle de espera.

```
docker ps --filter "name=p3-"
npm run db:logs
```

Si el problema persiste, reiniciar ambos contenedores:

```
npm run db:down
npm run db:up
```

9. Checklist de verificación final

Al terminar de seguir esta guía, cada miembro del equipo debe poder marcar las siguientes casillas. Si falta alguna, revisar el capítulo correspondiente antes de considerar el setup terminado.

9.1. Instalaciones

#	Elemento	¿Verificado?
1	node -v devuelve versión ≥ 20	si
2	npm -v devuelve versión ≥ 10	si
3	docker --version devuelve una versión	si
4	docker ps muestra cabecera (sin errores)	si
5	git --version devuelve versión ≥ 2.40	si
6	code --version devuelve una versión	si
7	VS Code tiene instaladas las 6 extensiones	si
8	Postman instalado y abierto al menos una vez	si

9.2. Repositorio y proyecto

#	Elemento	¿Verificado?
9	Invitación al repositorio aceptada	si
10	Repositorio clonado en la ruta recomendada	si
11	git branch -a muestra las 5 ramas remotas	si
12	git config user.name y user.email configurados	si
13	npm install ejecutado sin errores	si
14	.env creado a partir de .env.example	si
15	JWT_SECRET generado y pegado en .env	si
16	git status confirma que .env NO aparece	si

9.3. Docker y base de datos

#	Elemento	¿Verificado?
17	npm run db:up levanta contenedores sin errores	si
18	docker ps muestra p3-mongo y p3-mongo-express Up	si
19	p3-mongo aparece como (healthy)	si
20	http://localhost:8083 carga Mongo Express	si
21	Login con admin/admin entra al panel	si
22	npm run db:down apaga los contenedores	si
23	npm run db:up vuelve a levantarlos sin problema	si

9.4. GitFlow

#	Elemento	¿Verificado?
24	Posicionado en la rama feature/<propia>	si
25	Probado hacer un commit de prueba y push	si
26	Probado hacer un pull de develop en la rama personal	si
27	Entendido el flujo de Pull Request explicado en 6.5	si

Si todo está marcado...

El entorno está listo para empezar a programar. A partir de aquí el trabajo consiste en implementar las fases 2 a 6 descritas en el capítulo 7. ¡Buen trabajo!

10. Recursos y referencias

Lista de enlaces útiles consultados durante la fase de setup y que conviene tener a mano durante el desarrollo.

10.1. Documentación oficial

- **Node.js:** <https://nodejs.org/docs>
- **Express:** <https://expressjs.com>
- **Apollo Server:** <https://www.apollographql.com/docs/apollo-server>
- **GraphQL:** <https://graphql.org/learn>
- **MongoDB Node Driver:** <https://www.mongodb.com/docs/drivers/node>
- **Docker Compose:** <https://docs.docker.com/compose>
- **jsonwebtoken:** <https://github.com/auth0/node-jwebtoken>

10.2. Convenciones y buenas prácticas

- **Conventional Commits:** <https://www.conventionalcommits.org>
- **GitFlow original (Vincent Driessen):** <https://nvie.com/posts/a-successful-git-branching-model>
- **Twelve-Factor App (configuración por entorno):** <https://12factor.net/es/>

10.3. Enlaces internos del proyecto

- **Repositorio GitHub:** <https://github.com/ErickColl/FullStackAttack-Producto3>
- **Panel Mongo Express (local):** `http://localhost:8083`
- **Servidor GraphQL (local, tras arrancar):** `http://localhost:4000/graphql`

11. Prompts de IA utilizados

La rúbrica del Producto 3 exige documentar los prompts de inteligencia artificial empleados durante el desarrollo. El equipo ha usado asistentes de IA como apoyo puntual en varias fases: nunca para sustituir la comprensión del código, sino para acelerar tareas concretas, contrastar decisiones de arquitectura y resolver errores cuando la documentación oficial no era suficiente. Esta sección recoge los prompts más relevantes junto con su autor, el contexto en el que se utilizaron y el uso que se hizo del resultado obtenido.

11.1. Filosofía de uso de la IA en este proyecto

El equipo FullStackAttack ha adoptado tres reglas internas para el uso de asistentes de IA durante el desarrollo del Producto 3, coherentes con lo que se espera en un entorno profesional:

- Toda sugerencia de código generada por IA se lee, se entiende y se adapta al contexto del proyecto antes de commitearse. No se copia nada sin revisión.
- La IA se utiliza como herramienta de consulta rápida y de contraste de diseño (mentor técnico), no como generador automático de funcionalidades completas.
- Los prompts se registran honestamente, incluso los que no produjeron el resultado esperado, porque forman parte del aprendizaje documentable.

11.2. Tabla de prompts del equipo

Se recogen a continuación los prompts más significativos usados por cada miembro del equipo a lo largo del proyecto. La tabla está pensada para que el consultor pueda identificar rápidamente qué uso se dio a cada consulta y qué impacto tuvo en el código final. La tabla cubre las seis fases del desarrollo, desde la infraestructura inicial hasta el despliegue cloud. Se eligió la IA Chat GPT en todos los prompts.

Autor	Fase	Prompt resumido	Uso del resultado
Erick Coll	Fase 2.5	¿Cuál es la forma profesional de centralizar la configuración de variables de entorno en un proyecto Node.js con ES Modules, validando que todas las variables obligatorias existan antes de arrancar el servidor y devolviendo un objeto inmutable?	Se usó como referencia para diseñar <code>src/config/env.js</code> . La estructura final (<code>Object.freeze</code> , validación explícita al arrancar, excepción fatal si falta una variable) se inspiró en la respuesta, pero se reescribió por completo con los nombres y convenciones del proyecto.
Erick Coll	Fase 2.5	¿Cómo implementar un graceful shutdown en un servidor Express con Apollo Server y MongoDB que cierre ordenadamente las conexiones ante SIGINT y SIGTERM, evitando dejar recursos abiertos cuando el proceso termina?	La idea de registrar un handler por cada señal del sistema operativo y encadenar <code>apolloServer.stop()</code> con <code>closeMongo()</code> proviene de este prompt. El código final se adaptó al estilo del proyecto (JSDoc, logs con prefijos, <code>process.exit</code> con códigos distintos).
Erick Coll	Fase 3	Tengo en el Producto 2 una función que valida fechas con <code>new Date(isoString)</code> ; he detectado que acepta 2026-02-30 porque JavaScript lo autocorriga a marzo. ¿Cómo hago una validación estricta en JavaScript que rechace fechas imposibles?	Permitió identificar la técnica correcta: reconstruir el año, el mes y el día a partir del objeto <code>Date</code> y compararlos con los originales. El validador <code>esFechaValidaISO</code> de <code>src/utils/validators.js</code> implementa esta lógica y corrige así un bug que venía arrastrándose desde el P2.
Erick Coll	Fase 3	¿Cuál es el patrón idiomático en GraphQL con Apollo Server para manejar errores	Se utilizó para diseñar la jerarquía de clases en <code>src/utils/errors.js</code> (<code>AppError</code>

Autor	Fase	Prompt resumido	Uso del resultado
		tipados (validación, no encontrado, conflicto) que produzcan respuestas HTTP coherentes y que el frontend pueda distinguir fácilmente?	como base, subclases con statusCode y code). El código final es propio: la respuesta sirvió para validar que el enfoque elegido es el habitual en proyectos serios.
Carles Miguel	Fase 2	¿Cómo inicializar correctamente una conexión a MongoDB en un backend Node.js usando el driver nativo (sin Mongoose), con un patrón singleton para que el resto del código pueda acceder a la misma instancia mediante una función getDb?	Sirvió de guía para escribir la primera versión de src/config/db.js. El patrón singleton (guardar el cliente y la db en variables del módulo) se adoptó tal cual, pero las opciones de timeout, el logging y el cierre limpio se añadieron en la Fase 2.5.
Carles Miguel	Fase 4	Necesito migrar un model que usa un array en memoria (seleccionadas.push, seleccionadas.filter) a MongoDB manteniendo la misma interfaz pública. ¿Cómo traduzco cada operación del array a su equivalente en el driver nativo (insertOne, findOne, deleteOne, deleteMany)?	Plantilla de equivalencias array → MongoDB que se aplicó a los tres models. Útil especialmente para decidir cuándo usar deleteMany con operadores (\$in) frente a bucles de deleteOne en la función limpiarSeleccionesHuerfanas.
Carles Miguel	Fase 4	¿Cómo diseño un script de seed idempotente en MongoDB que inserte datos iniciales sólo si las colecciones están vacías, sin usar librerías externas y que sea seguro de ejecutar múltiples veces al arrancar el servidor?	Se utilizó para estructurar src/seed/seed.js. La comprobación con countDocuments antes de insertar y la separación entre crearIndice() y seedDatabase() vienen de ahí. Los datos concretos (usuarios y publicaciones) son los del Producto 2 del equipo.
Carles Miguel	Fase 4	En MongoDB, ¿cuál es la diferencia práctica entre validar una regla de unicidad a nivel de aplicación (con un findOne previo al insertOne) y hacerlo con un índice único en la colección? ¿Cuándo se recomienda combinar ambas?	Decidió añadir índices únicos sobre email (usuarios) y publicacionId (seleccionadas) en el script de seed. La validación en JavaScript se mantiene para dar errores de dominio claros, y el índice actúa como salvaguarda final a nivel de base de datos.
Jacobo	Fase 5	¿Cuál es el patrón estándar para implementar autenticación JWT en un backend Node.js con Apollo Server cuando solo el administrador necesita iniciar sesión? Quiero que el resto de	Confirmó la arquitectura final: middleware auth.js con extraerTokenDesdeRequest + obtenerUsuarioDesdeRequest (devuelve null si falla, no lanza), context de Apollo enriquecido en

Autor	Fase	Prompt resumido	Uso del resultado
		queries sigan siendo públicas y solo las mutations sensibles requieran token.	index.js, y guardas requireAuth/requireAdmin invocadas desde dentro de cada resolver protegido. La separación entre lo que no lanza (context) y lo que sí lanza (guards) viene de aquí.
Jacobo	Fase 5	Si en una mutation de login devuelvo distintos mensajes de error según el problema (email no existe, usuario sin rol admin, password incorrecta), ¿abro alguna vulnerabilidad?	Sí: enumeración de usuarios. La respuesta explicó que un atacante puede usar las distintas respuestas para deducir qué emails existen en el sistema. Por eso loginAdmin del modelo lanza siempre UnauthorizedError con el mismo mensaje genérico (Credenciales de administrador incorrectas) en los tres casos.
Jacobo	Fase 5	¿Cómo combino bcryptjs con un seed inicial de MongoDB que tiene que ser idempotente? Quiero que el admin se actualice si cambian las credenciales en .env, pero que no se duplique si ya existe.	Permitió diseñar la función asegurarAdminInicial con un patrón upsert manual: findOne por email, updateOne si existe (refrescando hash + datos) o insertOne si no existe. El bcrypt.hash se calcula siempre, así un cambio de ADMIN_PASSWORD en el .env se refleja en la base sin tener que vaciarla.
Jacobo	Fase 5	¿Qué información debe llevar el payload de un JWT y cuál es la diferencia entre meter el id en una clave personalizada (userId, idUsuario) o en el campo estándar sub?	Confirmó usar el campo estándar sub (subject) del estándar JWT en lugar de inventar un nombre propio. El payload final lleva sub (id del usuario), email y rol, nada más. La password ni se envía al token ni se loguea, y el secret se firma con HS256 usando la cadena del .env.
Erick Coll	Fase 6	Tengo una colección Postman con tests negativos que asumen extensions.code === "UNAUTHORIZED" pero el backend devuelve INTERNAL_SERVER_ERROR. He revisado el código y mis errores extienden de Error normal. ¿Apollo Server hace algo especial con los errores y por qué los enmascara?	Reveló la causa: Apollo enmascara automáticamente cualquier error que no extienda de la clase GraphQLError del paquete graphql. La solución (cambiar AppError extends Error por AppError extends GraphQLError y pasar extensions.code en super()) cabe en un solo fichero y arregla

Autor	Fase	Prompt resumido	Uso del resultado
			simultáneamente el código del error y el filtrado del stacktrace al cliente.
Erick Coll	Fase 6	¿Cuál es la mejor forma de gestionar un token JWT en una colección Postman para que se renueve automáticamente al hacer login y se inyecte como Authorization: Bearer en todas las requests posteriores, sin tener que copiarlo a mano cada vez que expira?	Inspiró el script post-response de la request 1. Auth → Login Admin: <code>pm.environment.set("token", responseJson.data.loginAdmin.token)</code> . El resto de requests usa Authorization: Bearer <code>{{token}}</code> y la magia funciona sin pegado manual. Es el patrón estándar y queda documentado en postman/README.md.
Erick Coll	Fase 6	Mi password de MongoDB Atlas tiene caracteres ! y @. He puesto la URI en MONGO_URI y el cliente falla con auth error sin más detalle. ¿Hay que codificar algo en la URI?	Confirmó que las URIs MongoDB siguen las reglas de URL-encoding estándar para userinfo: el carácter @ es el separador entre password y host, así que dentro de la password debe codificarse como %40, y otros caracteres especiales como ! como %21. Sin esto el parser rompe la URI en sitios incorrectos. La password <code>Dj!zt@hkZVY7HT5</code> quedó como <code>Dj%21zt%40hkZVY7HT5</code> en la URI final.
Erick Coll	Fase 6	Quiero desplegar mi backend Node.js + Apollo Server + MongoDB Atlas en CodeSandbox con URL pública. ¿Cómo configuro las variables de entorno (MONGO_URI, JWT_SECRET, ADMIN_PASSWORD) sin commitearlas al repositorio público?	Confirmó la ruta correcta: Editor → Repository → Env Variables dentro del Devbox. CodeSandbox guarda los valores cifrados y los inyecta en <code>process.env</code> al arrancar la VM. Tras configurarlas y reiniciar la VM, el backend conectó a Atlas a la primera y empezó a servir GraphQL en <code>https://2cw562-4000.csb.app/graphql</code> .

11.3. Consideraciones finales sobre el uso de IA

Los prompts documentados son representativos, no exhaustivos. En conversaciones largas con el asistente aparecen siempre mensajes intermedios de seguimiento, aclaraciones y correcciones mutuas que no tendría sentido transcribir literalmente aquí. Lo importante, desde el punto de vista de la rúbrica, es que la IA se ha usado como multiplicador de productividad y no como sustitutivo del trabajo técnico: todas las decisiones arquitectónicas, los nombres, la organización del código y la cobertura de casos límite han sido decisiones del equipo, no de la herramienta.

Algunos de los prompts más útiles del proyecto fueron precisamente los que destaparon problemas que el equipo no había detectado, como el bug del enmascarado de errores en Apollo Server (Fase 6, autor Erick) o la vulnerabilidad de enumeración de usuarios cuando se devuelven mensajes de error distintos según la causa del fallo de login (Fase 5, autor Jacobo). Esta capacidad de la IA para validar decisiones de seguridad y arquitectura ha sido especialmente valiosa cuando ningún miembro del equipo había trabajado antes con un patrón concreto.

También se han registrado los prompts que cubren la zona de despliegue (Atlas y CodeSandbox), porque son una parte sustancial del trabajo de la Fase 6 y conviene que el consultor entienda que la configuración cloud no se hizo a ciegas: cada decisión sobre URL-encoding de credenciales, sobre dónde guardar los secretos en CodeSandbox o sobre cómo configurar Network Access en Atlas pasó previamente por una validación en el asistente y se contrastó con la documentación oficial antes de aplicarla.